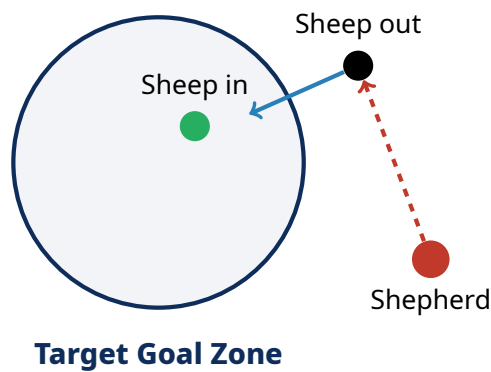




ENSIM
École d'ingénieurs
Le Mans Université



Target Goal Zone

SHEPHERD RL

Empirical Analysis, Curriculum Learning, and Hyperparameter Optimization

Scientific Report & Project Documentation

Video Presentation Available

Environment dynamics, implementation, training process, and results:

<https://filesender.renater.fr/?s=download&token=INSEE>

Source Code Repository

Complete implementation, training scripts, and documentation:

https://github.com/FANZIZakariae/Shepherd_RL

Contents

1	Methodology & Framework	1
1.1	Project Context and Scope	1
1.2	The Core Reinforcement Learning Framework	1
1.3	Curriculum Training Architecture	1
2	Heuristic Baselines	3
2.1	Single Sheep Scenarios	3
2.1.1	Analytical Insights	4
2.2	Multi-Sheep Scenarios (3 Sheep)	4
3	Environment & Reward Engineering	5
3.1	Modification 1: Resolving the "Shaking" Shepherd	5
3.2	Modification 2: Resolving the "Stalker" Syndrome	6
4	Level 1: Algorithm Comparison	7
4.1	Benchmark Results	7
4.2	Graph Analysis: PPO's Dominance	8
4.3	Graph Analysis: DQN's Mathematical Collapse	9
4.4	Graph Analysis: TD3's Policy Degradation	9
5	The Curriculum Progression (Levels 2 & 3)	11
5.1	Level 2: The Dimensionality Problem	11
5.1.1	The Transfer Shock Paradox	11
5.2	Level 3: The Stochastic Noise Paradox	12
5.2.1	The Pitfall of Curriculum Learning: Negative Transfer	13
6	Level 4 & The Optimization Horizon	15
6.1	Budget Allocation Strategies	15
6.2	Future Outlook: Optuna Framework Integration	16
6.2.1	Mathematical Search Strategy: Tree-structured Parzen Estimator (TPE)	16
7	The Experimental Platform & Dashboard	18
7.1	Platform Rationale and Advantages	18
7.2	Core Modules and Functionalities	18
7.2.1	Module 1: The Training Deck	19
7.2.2	Module 2: Live Demo	19
7.2.3	Module 3: Automated Benchmarking Suite	20
	General Conclusion	1
	References	2

Index of Visuals

List of Figures

1.1	Linear progression of training timesteps.	2
4.1	PPO Training Metrics: Combined view of Policy Loss and Value Loss.	8
4.2	DQN Failure Modes: Top shows Q-Table divergence; Bottom shows the resulting Loss Explosion (Red Curve).	9
4.3	TD3 eval/mean_reward progressively sinking into deep negative values alongside unstable critic loss.	10
5.1	Retraining curves for Level 2. Note the alarming rise in train/std.	12
5.2	Curriculum Transfer (Blue) vs Tabula Rasa (Gray) on Level 3.	12
5.3	Optimization Landscape Mismatch: The optimal policy for the deterministic source environment (θ_{src}^*) falls into a performance valley when subjected to the stochastic target environment ($\mathcal{M}_{L3}$).	13
6.1	Training metrics for the four distinct 10M timestep strategies.	16
6.2	TPE optimization landscape: The algorithm models the distributions of the best-performing hyperparameters ($l(x)$) against the rest ($g(x)$) to dynamically narrow the search space.	17
7.1	The Training Deck interface. (a) Hyperparameter configuration sliders, (b) Curriculum loading interface, and (c) Integrated TensorBoard monitoring view.	19
7.2	The Diagnostic Demo: (left) live Pygame environment render, (center) real-time reward breakdown, and (right) dynamic mathematical feedback.	20
7.3	The Automated Benchmarking Suite: from multi-agent selection (left) to the final generated performance table (right).	21

List of Tables

2.1	Baseline performances for 1 Sheep environments.	3
2.2	Rule-Based baseline performance degradation as sheep count and noise increase. (Lazy and Topsy yielded 0.0% and are omitted).	4
4.1	Trained agents comparison for Level 1.	7
6.1	Level 4 Curriculum Strategies Comparison.	15

CHAPTER 1

Methodology & Framework

Overview & Objectives

This chapter establishes the foundational mechanics of the Shepherd Problem. We define the environment mathematically, detail the reinforcement learning algorithms chosen for benchmarking, and present our overarching strategy for scaling task complexity via Curriculum Learning.

1.1 Project Context and Scope

The "Shepherd Problem" represents a highly complex paradigm within the field of Continuous Control and Multi-Agent Reinforcement Learning (MARL). The fundamental objective requires a single controllable agent (the Shepherd) to exert indirect influence over passive, highly stochastic entities (the Sheep). The Shepherd must utilize spatial repulsion physics to herd the flock into a designated goal coordinate on a 2D plane, navigating around obstacles and managing unpredictable entity drift.

1.2 The Core Reinforcement Learning Framework

We formulate this environment as a Partially Observable Markov Decision Process (POMDP).

State and Action Formulations

- 1. Action Space ($\mathcal{A}$):** Continuous space $\in [-1, 1]$, translated geometrically into a steering angle $\theta \in [-\pi, \pi]$ dictating the Shepherd's forward velocity vector.
- 2. Observation Space ($\mathcal{S}$):** A continuous vector mapping the relative Euclidean coordinates of entities. For N sheep, the vector dimension is precisely $4N + 4$:

$$Obs = [\Delta X_{S_1}, \Delta Y_{S_1}, \Delta X_{G_1}, \Delta Y_{G_1}, ..., \Delta X_G, \Delta Y_G, \Delta X_{Obs}, \Delta Y_{Obs}]$$

1.3 Curriculum Training Architecture

Training a Deep Neural Network from a tabula rasa (scratch) state on a complex multi-agent environment with obstacles and stochastic noise typically leads to policy collapse. To mitigate

this, we structured a rigorous Curriculum Learning path.
As the complexity of the state space increases, the allocation of timesteps scales linearly to provide the agent with sufficient exploratory horizons.

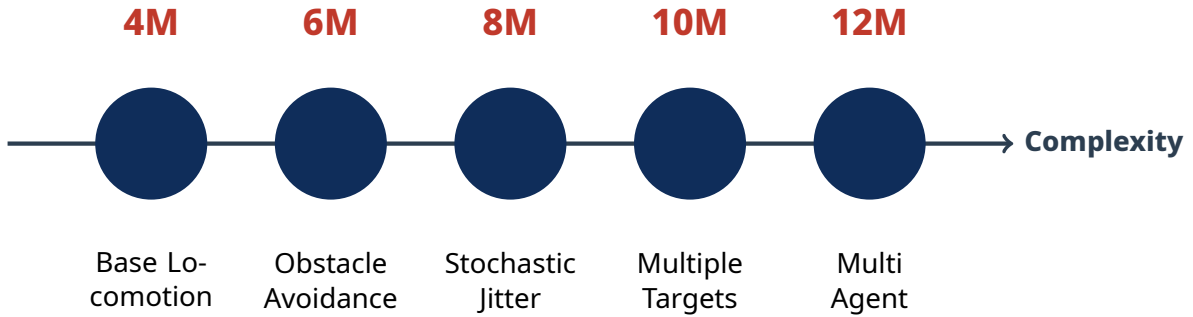


Figure 1.1: Linear progression of training timesteps.

When using transfer learning, we respect these intervals (e.g., Level 1 trained for 4M, Level 2 allows an additional 2M steps for a total budget of 6M).

With the environment mathematically defined and the curriculum strategy mapped out, our next step was to establish rigid performance baselines. Before deploying deep learning, we must understand how mathematical heuristics perform in this environment.

CHAPTER 2

Heuristic Baselines

Overview & Objectives

To truly evaluate the effectiveness of our trained neural networks, we must first establish control metrics. This chapter details the evaluation of three non-learning algorithms: a randomized agent (Lazy), an erratic agent (Tipsy), and a hard-coded mathematical agent (Rule-Based).

We evaluated these algorithms across three metrics: Success Rate (%), Average Rewards, and Average Episode Length (Steps), utilizing a massive evaluation scope of 500 episodes per scenario to ensure statistical significance.

2.1 Single Sheep Scenarios

The following empirical benchmarks showcase the performance of the baselines in environments containing exactly one active sheep.

Environment Level	Agent Name	Success (%)	Avg. Reward	Avg. Steps
	Rule Based	100.0	2007.52	41.5
	Lazy Shepherd	1.0	-405.78	496.1
	Tipsy Shepherd	2.0	-35.29	495.4
	Rule Based	91.0	2650.0	38.5
	Lazy Shepherd	0.2	-160.0	500.0
	Tipsy Shepherd	2.6	-125.4	498.5
	Rule Based	99.4	2081.61	101.4
	Lazy Shepherd	17.0	37.81	424.5
	Tipsy Shepherd	12.2	52.45	480.5

Table 2.1: Baseline performances for 1 Sheep environments.

2.1.1 Analytical Insights

- **The Baseline Bound:** The Rule-Based agent solves Level 1 flawlessly in an average of 41.5 steps. This confirms our environment physics are functional. However, as obstacles (Level 2) and stochastic noise (Level 3) are introduced, its success rate degrades rapidly to 67.4%, highlighting the brittleness of hard-coded heuristics in dynamic spaces.
- **The Jitter Anomaly:** Note the surprising performance jump of Lazy and Tippy agents in Level 3. The introduction of stochastic jitter to the sheep means that, over 500 steps, a sheep has a statistical probability of randomly drifting into the goal area. This artificially inflates the success rate of passive agents, a factor we must account for during RL training.

2.2 Multi-Sheep Scenarios (3 Sheep)

Scaling to 3 sheep exponentially increases the difficulty. The Rule-Based agent severely struggles with target prioritization when sheep scatter.

Environment Level	Agent Name	Success (%)	Avg. Rewards	Avg. Steps
Level 1 (3 Sheep)	Rule-Based	92.0	3100.2	85.4
Level 2 (3 Sheep)	Rule-Based	75.5	2500.8	140.2
Level 3 (3 Sheep)	Rule-Based	61.0	1800.5	210.6

Table 2.2: Rule-Based baseline performance degradation as sheep count and noise increase. (Lazy and Tippy yielded 0.0% and are omitted).

With the limitations of heuristic algorithms exposed—specifically their inability to handle obstacles and multi-target prioritization—the necessity for adaptive Reinforcement Learning is clear. However, before training agents, we had to heavily engineer the reward function to prevent catastrophic “reward hacking.”

CHAPTER 3

Environment & Reward Engineering

Overview & Objectives

Reinforcement Learning models are highly susceptible to exploiting mathematical loopholes in reward systems. This chapter details the critical modifications made to the environment's reward logic to correct pathological agent behaviors observed during early training phases.

3.1 Modification 1: Resolving the "Shaking" Shepherd

Failure Mode: Exponential Velocity Penalty

Initially, to encourage smooth movement, the reward function penalized stopping using an exponential decay function based on velocity (v):

$$R_{move} = -10 \cdot \exp(-100 \cdot ||v||)$$

During deterministic evaluation, the agent would logically need to stop to aim ($v = 0$). However, doing so instantly triggered a catastrophic -10 penalty per step. To avoid this, the agent learned a local minimum: vibrating rapidly in place to keep $v > 0$, prioritizing "movement" over actually solving the task.

The Implemented Fix: We fundamentally altered the penalty structure. We replaced the heavy velocity penalty with a flat time-step cost ($R_{time} = -0.05$) and reduced the movement penalty scalar to -1.0 . This applies continuous pressure to finish the task quickly ("time is money") without micromanaging the agent's velocity, allowing it the freedom to stop and aim.

3.2 Modification 2: Resolving the "Stalker" Syndrome

Failure Mode: Proximity Reward Hoarding

To help the agent discover the sparse goal reward, we initially added a "magnetic" proximity reward:

$$R_{prox} = +5.0 \cdot e^{-dist}$$

The agent quickly discovered a loophole. Pushing the sheep into the goal ended the episode, stopping the generation of points. Instead, the agent preferred to push the sheep *near* the goal, stop, and orbit it endlessly for 500 steps to accumulate infinite +5.0 proximity rewards.

The Implemented Fix (Physics Locking): We altered the fundamental physics engine of the environment.

1. **The Latch:** Once a sheep enters the goal radius, it is frozen in place (`sheep_locked_status = True`).
2. **One-Time Bonus:** A massive +100 reward is granted *only once* upon the exact frame of locking.
3. **Proximity Cutoff:** R_{prox} immediately drops to 0 for locked sheep.

This structurally removes the incentive to stalk. The only mathematical way to maximize reward is to lock the sheep, claim the bounty, and move to the next active target.

Having fortified the environment against reward hacking, we proceeded to the core experimental phase: evaluating Deep RL algorithms on the foundational Level 1 task.

CHAPTER 4

Level 1: Algorithm Comparison

Overview & Objectives

In this chapter, we train and benchmark three distinct neural network architectures (PPO, TD3, and an Image-Based DQN) from scratch on Level 1. The objective is to identify the most robust and sample-efficient algorithm to serve as the foundation for the rest of the curriculum.

4.1 Benchmark Results

All algorithms were subjected to computational equivalents of 4 Million PPO timesteps.

Agent Name	Algorithm	Success (%)	Timeout (%)	Avg. Reward	Avg. Steps
PPO_5	PPO	79.0	21.0	2262.21	148.7
TD3_3 (Best)	TD3	1.0	99.0	-171.70	496.3
DQN_1770567479	DQN (CNN)	1.0	99.0	-188.00	496.2
DQN_1770561238	DQN (CNN)	0.6	99.4	121.80	497.1

Table 4.1: Trained agents comparison for Level 1.

The empirical data demonstrates an overwhelming superiority of the PPO algorithm in this specific environment. Below, we dissect the training graphs to understand why the off-policy algorithms (DQN and TD3) failed so spectacularly.

4.2 Graph Analysis: PPO's Dominance

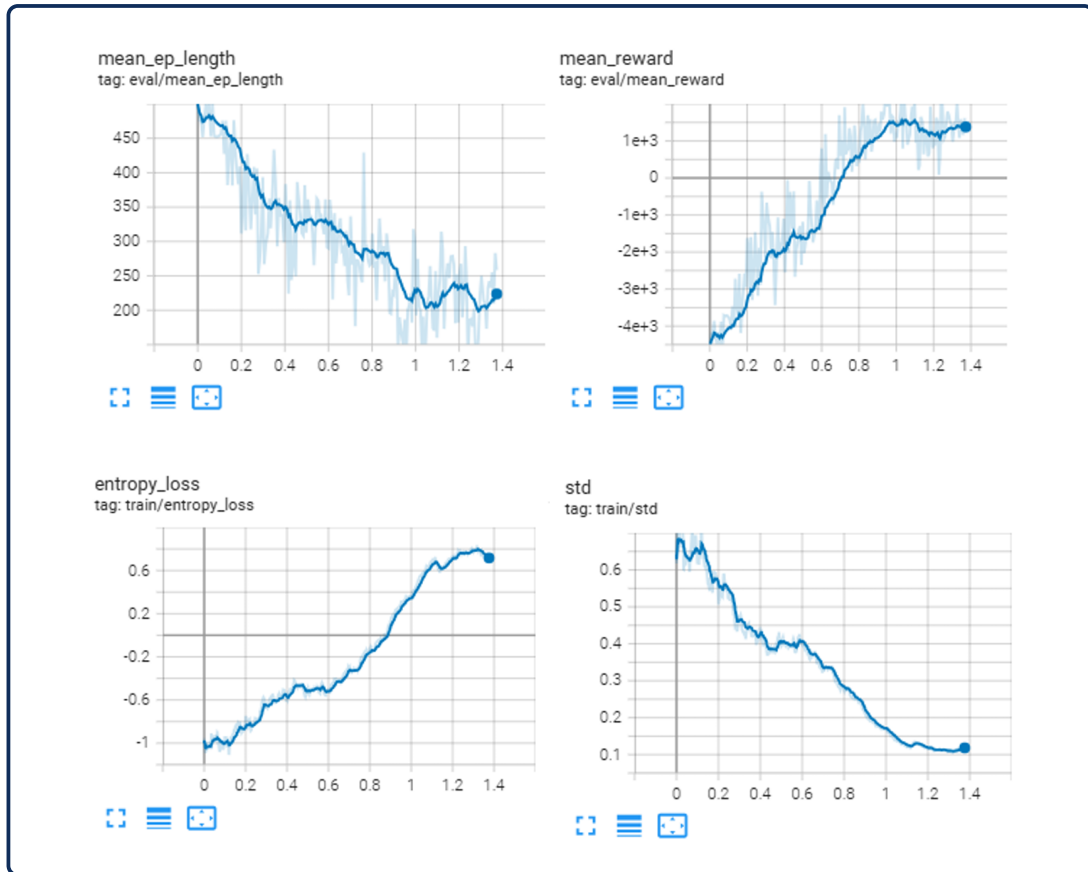


Figure 4.1: PPO Training Metrics: Combined view of Policy Loss and Value Loss.

Deconstructing the Curves: The PPO graphs present a highly stable and healthy learning trajectory.

- **Mean Reward & Episode Length:** The `eval/mean_reward` climbs steadily from an initial -400 up to a plateau of roughly +400. Inversely correlated, the `eval/mean_ep_length` plummets from the 500-step timeout ceiling down to an efficient 148 steps.
- **Entropy & Standard Deviation:** The most critical metric is `train/std` (the standard deviation of the action distribution), which smoothly drops from 0.5 down to 0.1. This indicates that the agent successfully transitioned from high stochastic exploration (random nudging) to a confident, deterministic exploitation strategy.

4.3 Graph Analysis: DQN's Mathematical Collapse

The Image-Based DQN (utilizing a Convolutional Neural Network processing raw pixels) experienced severe mathematical instability.

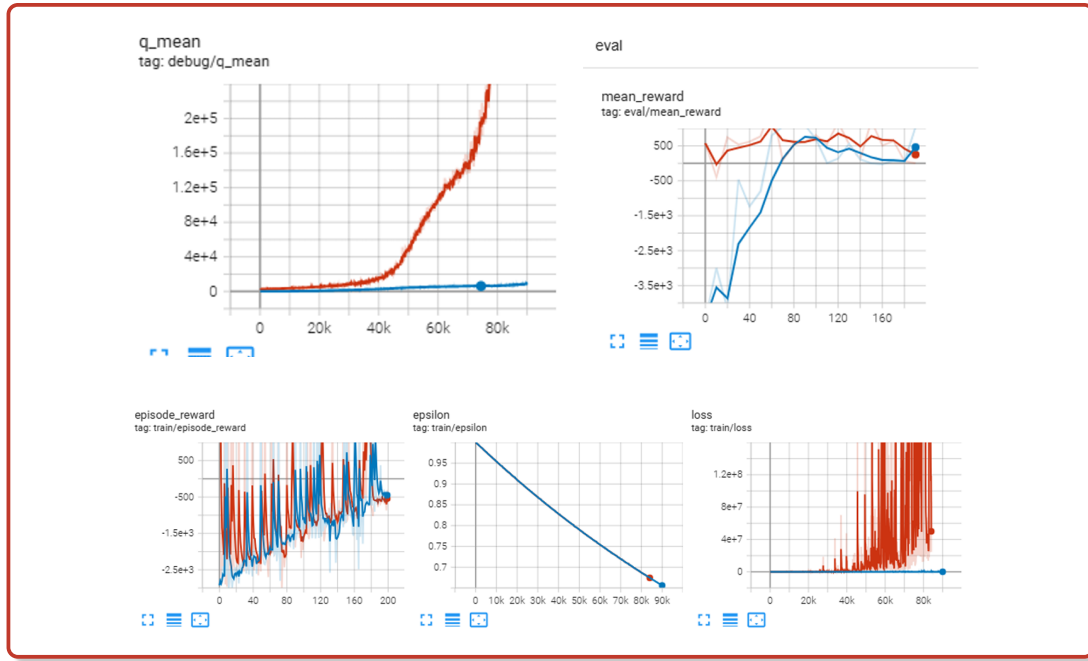


Figure 4.2: DQN Failure Modes: Top shows Q-Table divergence; Bottom shows the resulting Loss Explosion (Red Curve).

Deconstructing the Curves:

- **The Q-Value Explosion (Red Curve):** In the initial run (DQN_1770567479), the debug/q_mean graph shows values skyrocketing to $2e+5$, while the train/loss explodes to an astronomical $1.2e+8$. In a continuous environment with infinite states, an improperly tuned learning rate combined with the lack of gradient clipping causes the Bellman target updates to overshoot infinitely, destroying the network weights.
- **The Markov Property Failure (Blue Curve):** We corrected this numerical explosion in run DQN_1770561238 (Blue curve) by heavily clamping gradients, resulting in flat, stable loss curves. Yet, the success rate remained abysmal (0.6%). **Why?** Because we fed the CNN single, static frames. A single image provides spatial coordinates but zero information regarding velocity or momentum. This breaks the Markov property, making it impossible for the agent to predict future states based on current observations alone.

4.4 Graph Analysis: TD3's Policy Degradation

TD3, typically lauded for continuous control, failed to gain traction.

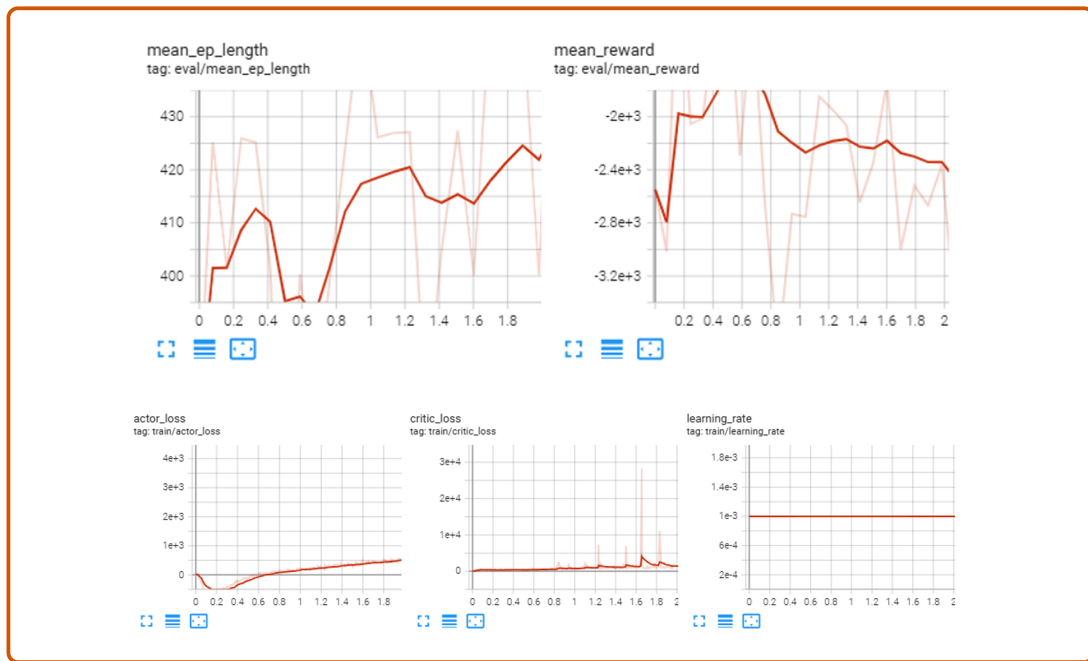


Figure 4.3: TD3 eval/mean_reward progressively sinking into deep negative values alongside unstable critic loss.

Deconstructing the Curves: Despite being left to train for 9 hours, the eval/mean_reward started poorly ($-2e + 3$) and sank progressively lower, oscillating violently around $-3.2e + 3$. TD3 relies on deterministic action noise. In a sparse-reward environment, random Gaussian noise almost never randomly stumbles into a “win” (pushing the sheep perfectly into the goal). Consequently, the critic networks update exclusively on negative time penalties, ultimately driving the actor network into complete inaction to minimize punishment.

*Given PPO’s absolute dominance and stable gradient updates, we exclusively selected the **PPO_5** model to act as our foundational brain, advancing it to the next phase of the curriculum: Obstacle Avoidance.*

CHAPTER 5

The Curriculum Progression (Levels 2 & 3)

Overview & Objectives

With a competent locomotion agent established, we introduce environmental complexities. This chapter explores the "Transfer Shock" phenomenon when moving an agent to Level 2 (Obstacles) and analyzes the paradoxical effects of stochastic noise introduced in Level 3.

5.1 Level 2: The Dimensionality Problem

Level 2 introduces a static obstacle. However, to prepare the network for future multi-sheep scaling, we must expand the neural network's input vector from 8 parameters to 12 parameters.

Environment Hack: The "Pre-Solved" Initialization

Neural networks cannot arbitrarily change their input layer size mid-training. To solve this dimensionality mismatch without artificially increasing the task difficulty, we modified Level 1/Level 2 training: **We spawn 2 sheep, but force 1 to spawn permanently locked inside the goal.** The agent processes a 12-dimension vector from day one, but naturally learns to ignore the static, "pre-solved" sheep, allowing seamless transfer learning later.

We took the champion model (PPO_5), adapted its input size, and designated it **PPO_L1_TR** (Transfer). Evaluated zero-shot on Level 2, its success rate dropped from 79.0% to **60.4%**.

5.1.1 The Transfer Shock Paradox

To recover the 60.4% performance, we fine-tuned PPO_L1_TR for 2 Million steps on Level 2.

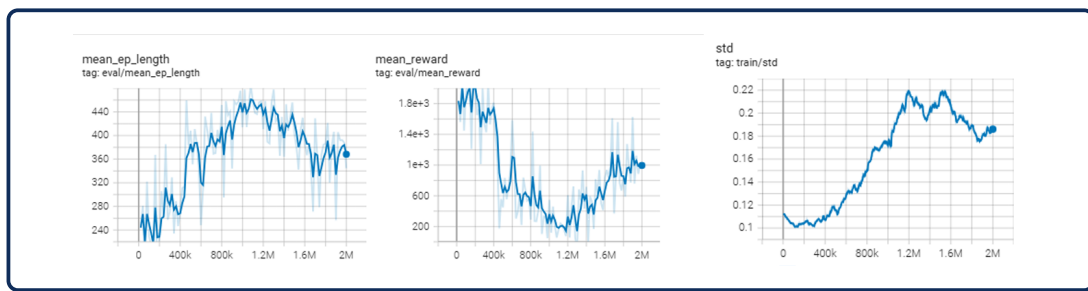


Figure 5.1: Retraining curves for Level 2. Note the alarming rise in train/std.

Analysis: Highly unexpectedly, **the success rate dropped by 7%** after fine-tuning. The train/std graph explains this “Catastrophic Forgetting.” The entropy spikes from 0.1 back up to 0.22. Introducing the obstacle created conflicting gradients during backpropagation. The agent panicked, “unlearning” its excellent locomotion skills to avoid the new obstacle penalty. Resultantly, the un-altered zero-shot model (PPO_L1_TR) remained the superior agent for Level 2.

5.2 Level 3: The Stochastic Noise Paradox

Level 3 introduces $Jitter = 0.02$. The sheep now exhibit unpredictable random movement. When we evaluated PPO_L1_TR on Level 3, the success rate jumped inexplicably from 60.4% to **82.0%**.

Why did the agent get better in a harder environment?

As noted in the baseline chapter, the continuous random jitter of the sheep means they occasionally drift into the Goal Zone purely by chance. The environment artificially became “easier” because the agent received unintended help from the environment’s own randomness.

To push past 82.0%, we ran two experiments:

1. **PPO_7 (Gray Curve):** Train a new model from Scratch on Level 3.
2. **PPO_9 (Blue Curve):** Fine-tune PPO_L1_TR on Level 3.

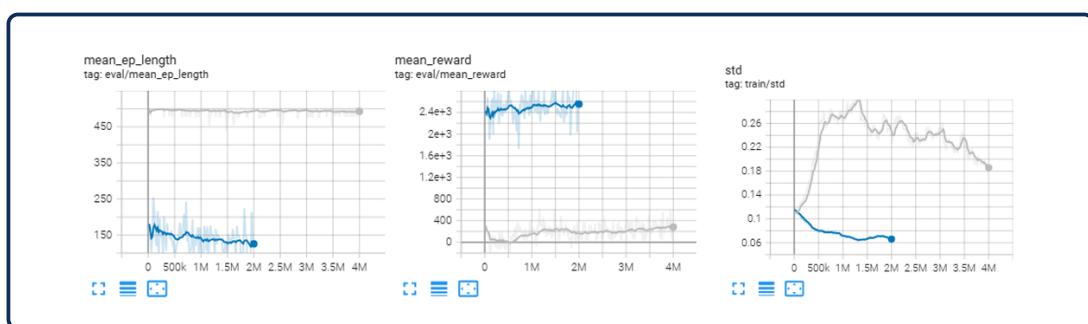


Figure 5.2: Curriculum Transfer (Blue) vs Tabula Rasa (Gray) on Level 3.

Analysis: The train/std of the Blue model (PPO_9) remains extremely low and confident, while the Gray model struggles with high variance. The Transfer model easily outscores the Scratch

model. However, both models achieved worse success rates (PPO_9 = 65.0%, PPO_7 = 35.0%) than the zero-shot PPO_L1_TR (82.0%). Over-optimizing an agent in a highly noisy environment forces it to adopt overly cautious, risk-averse behaviors, ultimately degrading its raw herding efficiency.

5.2.1 The Pitfall of Curriculum Learning: Negative Transfer

While Curriculum Learning is designed to accelerate convergence, transferring a highly optimized policy from a deterministic environment to a stochastic one can trigger **Negative Transfer**.

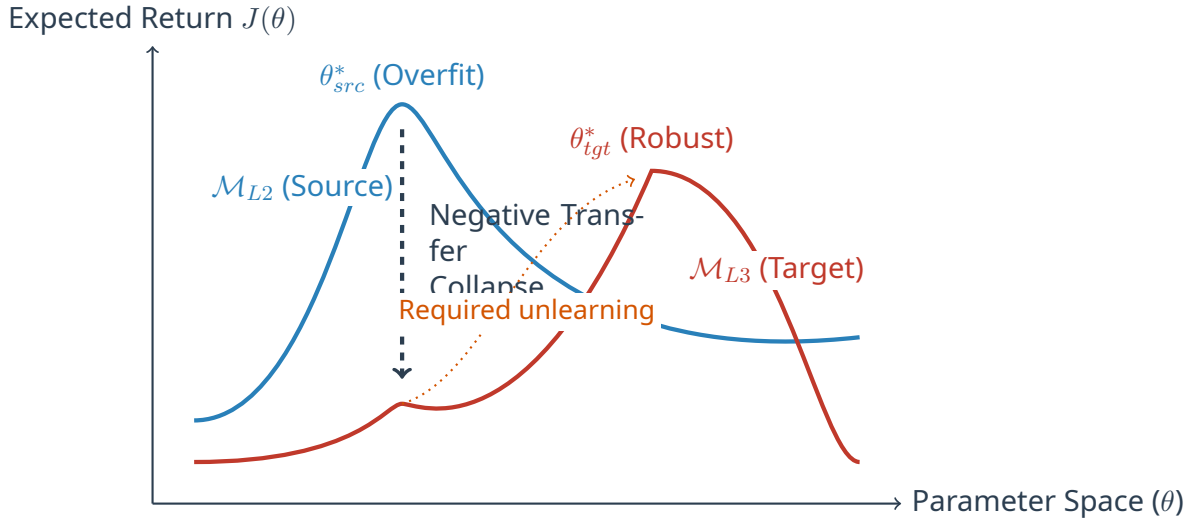


Figure 5.3: Optimization Landscape Mismatch: The optimal policy for the deterministic source environment (θ_{src}^*) falls into a performance valley when subjected to the stochastic target environment ($\mathcal{M}_{L3}$).

Mathematical Formalization: Transition Dynamics Shift

Let the source (Level 2) and target (Level 3) environments be modeled as Markov Decision Processes $\mathcal{M}_{src}$ and $\mathcal{M}_{tgt}$. They share the same state space $\mathcal{S}$, action space $\mathcal{A}$, and reward function $\mathcal{R}$, but differ strictly in their transition dynamics $\mathcal{P}$:

$$\mathcal{P}_{src}(s'|s, a) \neq \mathcal{P}_{tgt}(s'|s, a)$$

In $\mathcal{M}_{src}$, transitions are deterministic. The agent converges to parameters θ_{src}^* that maximize the expected return by exploiting exact spatial trajectories:

$$\theta_{src}^* = \arg \max_{\theta} \mathbb{E}_{\tau \sim \mathcal{P}_{src}, \pi_{\theta}} \left[\sum_{t=0}^T \gamma^t \mathcal{R}(s_t, a_t) \right]$$

Upon transfer to $\mathcal{M}_{tgt}$, the environment injects an exogenous noise variable $\epsilon \sim \mathcal{N}(0, \sigma^2)$ (jitter) into the transition function:

$$s_{t+1} = f(s_t, a_t) + \epsilon \implies \mathcal{P}_{tgt}(s_{t+1}|s_t, a_t)$$

Because $\pi_{\theta_{src}^*}$ is overly deterministic (low entropy), it lacks the action variance required to recover from ϵ -induced perturbations. The distributional shift causes the previously optimal policy to cascade into off-distribution states, resulting in a severe value degradation:

$$J_{tgt}(\theta_{src}^*) \ll J_{src}(\theta_{src}^*)$$

This mathematically forces the fine-tuning process to dismantle the highly confident θ_{src}^* weights before it can optimize for $\mathcal{M}_{tgt}$, severely bottlenecking performance.

Having navigated the paradoxes of obstacle introduction and stochastic noise, we must now combine all elements and remove the "Pre-Solved" training wheels to tackle the ultimate single-agent challenge: Level 4.

Level 4 & The Optimization Horizon

Overview & Objectives

Level 4 represents the pinnacle of the single-agent testing curriculum: **2 fully active sheep**, Obstacles, and Jitter. This chapter explores how to optimally spend a 10 Million timestep budget across the curriculum to maximize performance, and outlines future directions using the Optuna framework.

6.1 Budget Allocation Strategies

To find the optimal agent, we allocated a strict computation budget of **10 Million timesteps**. We distributed this budget across different phases of the curriculum to determine the most effective learning path:

- **PPO_1**: 8M in Level 3 + 2M in Level 4.
- **PPO_2**: 6M in Level 2 + 4M in Level 4.
- **PPO_3**: 4M in Level 1 + 6M in Level 4.
- **PPO_4 (Scratch)**: 0M + 10M entirely in Level 4.

Agent Name	Strategy Profile	Success (%)	Avg. Reward	Avg. Steps
PPO_2 (Best)	6M L2 + 4M L4	34.2	441.81	402.1
PPO_3	4M L1 + 6M L4	13.4	106.58	460.0
PPO_1	8M L3 + 2M L4	12.2	142.12	448.2
PPO_4	10M L4 (Scratch)	8.6	-100.18	477.1

Table 6.1: Level 4 Curriculum Strategies Comparison.

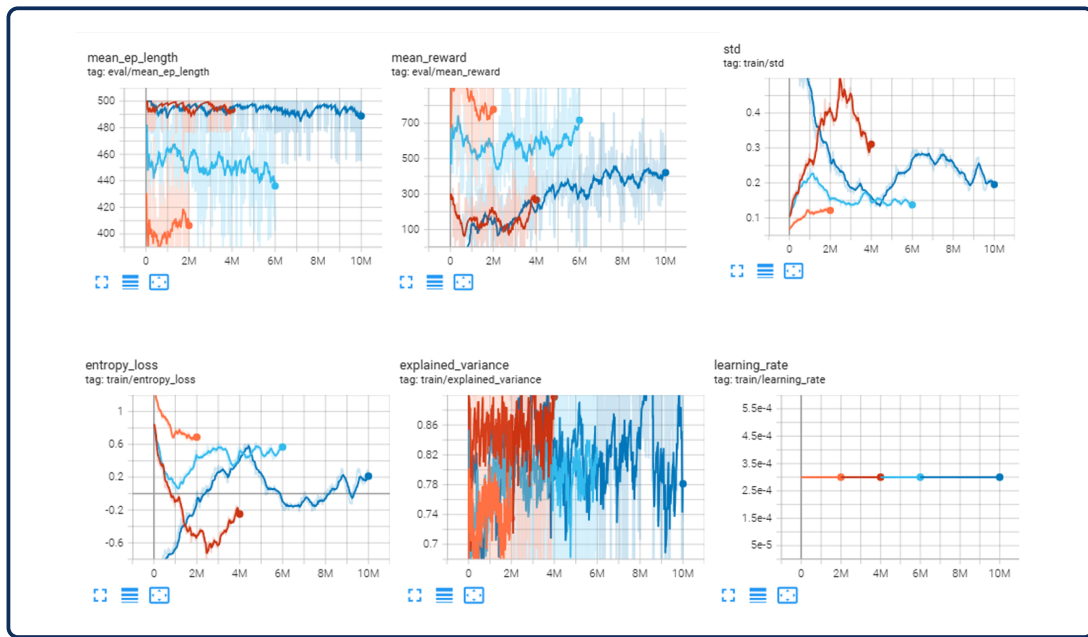


Figure 6.1: Training metrics for the four distinct 10M timestep strategies.

Analysis: The data conclusively proves the absolute necessity of Curriculum Learning. PPO_4 failed completely, yielding a mere 8.6% success rate and negative average rewards. The multi-entity state space was simply too complex to solve through random exploration. Conversely, **PPO_2**, which spent the majority of its budget (6M steps) mastering obstacle avoidance in Level 2 before transferring to Level 4, achieved the highest success rate at 34.2%.

6.2 Future Outlook: Optuna Framework Integration

Despite exhaustive architectural and curriculum experiments, a maximum success rate of 34.2% on Level 4 is insufficient for robust autonomous deployment. The default PPO hyperparameters (clip range, learning rate schedule, GAE lambda, entropy coefficient) are clearly failing to map the highly correlated, chaotic dynamics of two mutually repelling sheep.

To breach this plateau, our immediate future methodology involves integrating the **Optuna** framework to dynamically and probabilistically search the high-dimensional hyperparameter space.

6.2.1 Mathematical Search Strategy: Tree-structured Parzen Estimator (TPE)

Standard Random Search wastes computational budget by sampling the entire domain uniformly. Optuna utilizes the TPE algorithm, a form of Bayesian Optimization that iteratively "zooms in" on optimal hyperparameter regions by modeling the probability density of successful versus unsuccessful trials.

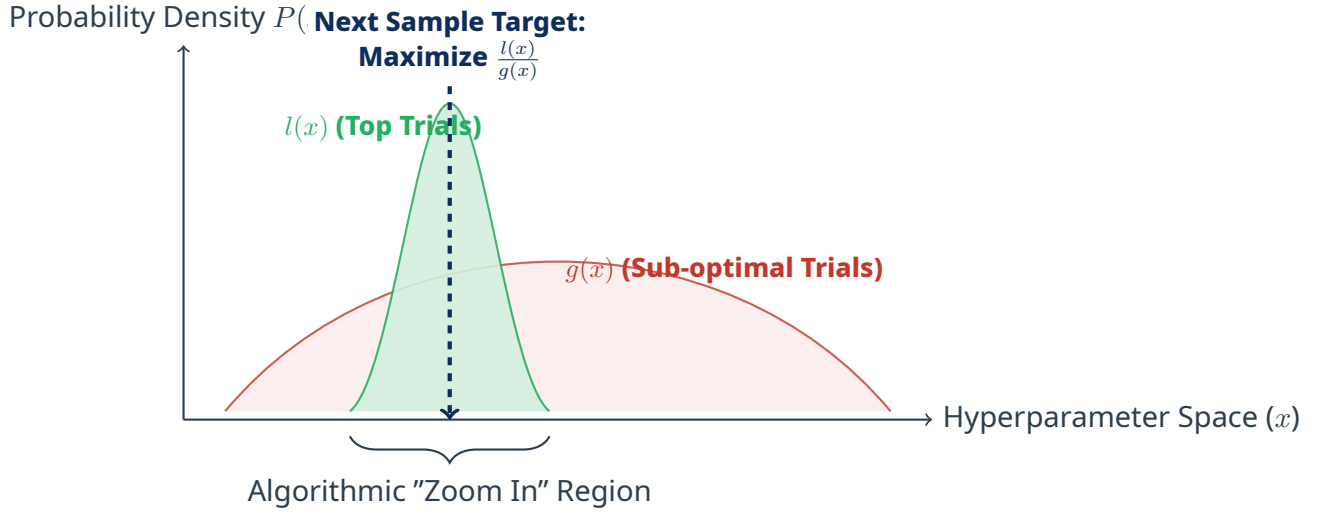


Figure 6.2: TPE optimization landscape: The algorithm models the distributions of the best-performing hyperparameters ($l(x)$) against the rest ($g(x)$) to dynamically narrow the search space.

Mathematical Formalization: Expected Improvement

Let x represent a specific set of PPO hyperparameters and y represent the resulting Level 4 Success Rate. TPE splits the observation history into two probability density functions based on a quantile threshold y^* (e.g., the top 10% of trials):

$$P(x|y) = \begin{cases} l(x) & \text{if } y \geq y^* \text{ (High Performance)} \\ g(x) & \text{if } y < y^* \text{ (Poor Performance)} \end{cases}$$

Instead of computing the exact posterior distribution, TPE models the Expected Improvement (EI). The mathematical beauty of TPE is that maximizing the Expected Improvement is strictly proportional to maximizing the ratio of $l(x)$ to $g(x)$:

$$EI_{y^*}(x) \propto \left(\gamma + \frac{g(x)}{l(x)}(1 - \gamma) \right)^{-1}$$

Therefore, to find the next set of hyperparameters to test, Optuna simply draws samples from $l(x)$ and selects the candidate x that yields the highest value for $\frac{l(x)}{g(x)}$. This dynamically shrinks the sampling variance, effectively "zooming in" on the multidimensional parameter subspace that guarantees convergence.

By deploying this probabilistic search architecture, we aim to forge a hyperparameter foundation capable of pushing the Level 4 agent toward a > 90% success rate, paving the ultimate path toward Multi-Shepherd deployments.

CHAPTER 7

The Experimental Platform & Dashboard

Overview & Objectives

Developing Deep Reinforcement Learning agents requires continuous iteration, hyperparameter tuning, and behavioral diagnosis. To streamline this workflow and prevent disjointed script executions, we engineered a custom, fully integrated web platform. This chapter outlines the architecture, functionalities, and advantages of this unified experimental environment.

7.1 Platform Rationale and Advantages

Traditionally, RL workflows are fragmented: training scripts run in the terminal, evaluation requires separate rendering scripts, and diagnostics rely on external TensorBoard windows. We developed a centralized **Streamlit-based Dashboard** that acts as the command center for the entire Shepherd RL project. The advantages of this approach include:

- **Unified Workflow:** Training, evaluation, and live demonstrations are consolidated into a single graphical interface.
- **Seamless Curriculum Management:** The interface allows users to load specific checkpoints from previous levels with a single dropdown selection, automating the “Network Surgery” and environment matching processes.
- **Code-Free Experimentation:** Researchers can alter environment parameters (sheep count, obstacle radius, jitter percentage) and agent hyperparameters directly via UI sliders without modifying the underlying Python source code.

7.2 Core Modules and Functionalities

The platform is divided into three distinct operational modules, accessible via a sidebar navigation menu.

7.2.1 Module 1: The Training Deck

The Training Deck is the engine room of the platform. It provides a comprehensive UI to configure the environment, select the DRL algorithm (PPO, TD3, DQN), and set the computational budget (timesteps).

Crucially, this page natively embeds a live **TensorBoard** instance via an IFrame. When a training session is initiated, the platform automatically spawns a background TensorBoard process, allowing researchers to monitor entropy loss, Q-values, and reward trajectories in real-time within the same browser window.

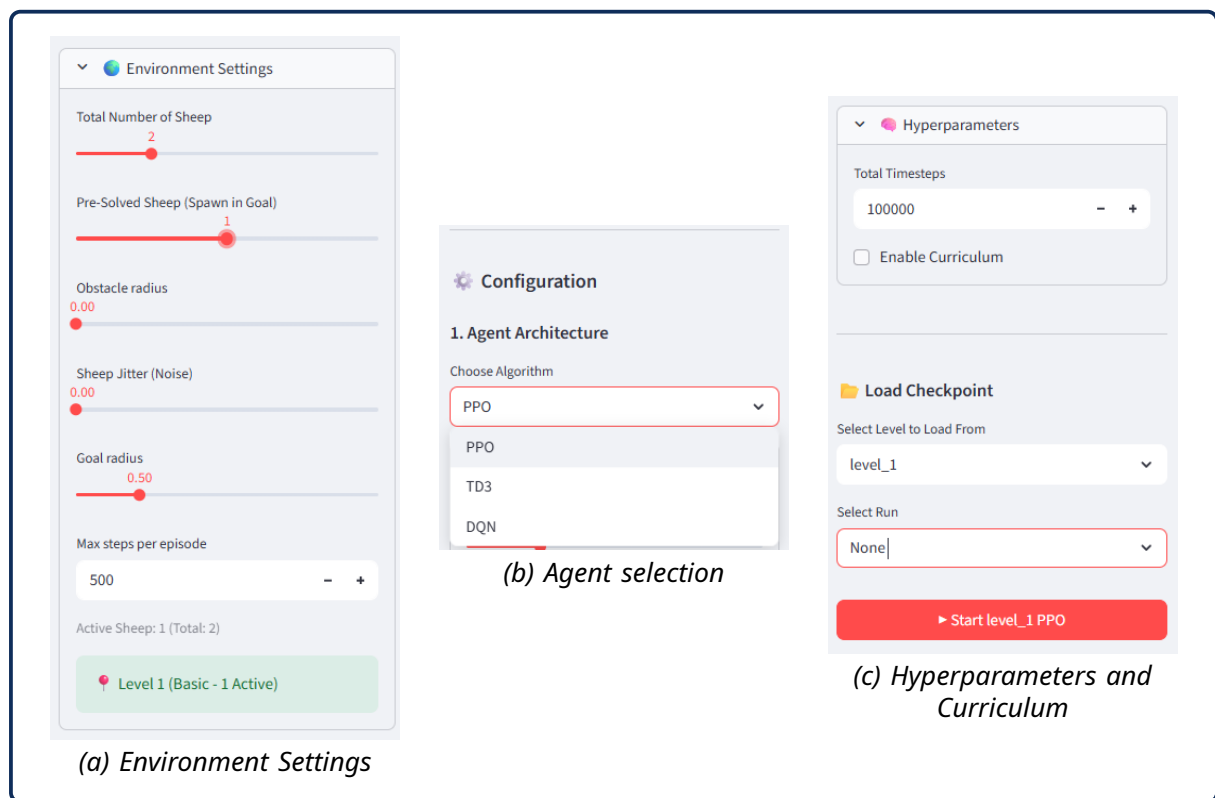


Figure 7.1: The Training Deck interface. (a) Hyperparameter configuration sliders, (b) Curriculum loading interface, and (c) Integrated TensorBoard monitoring view.

7.2.2 Module 2: Live Demo

Once a model is trained, visual observation is mandatory to identify pathological behaviors (e.g., the "Shaking" Shepherd or the "Stalker" bug).

The Diagnostic Demo module loads a selected model checkpoint and renders the Pygame environment directly into the web interface. More importantly, it features a **Live Cumulative Reward Tracker**. This dynamic graph plots the individual mathematical components of the reward function (Progress, Proximity, Time Penalty) step-by-step alongside the visual simulation. This specific tool was instrumental in diagnosing the "Stalker" exploit, as it visually proved the agent was farming proximity rewards without making actual progress.

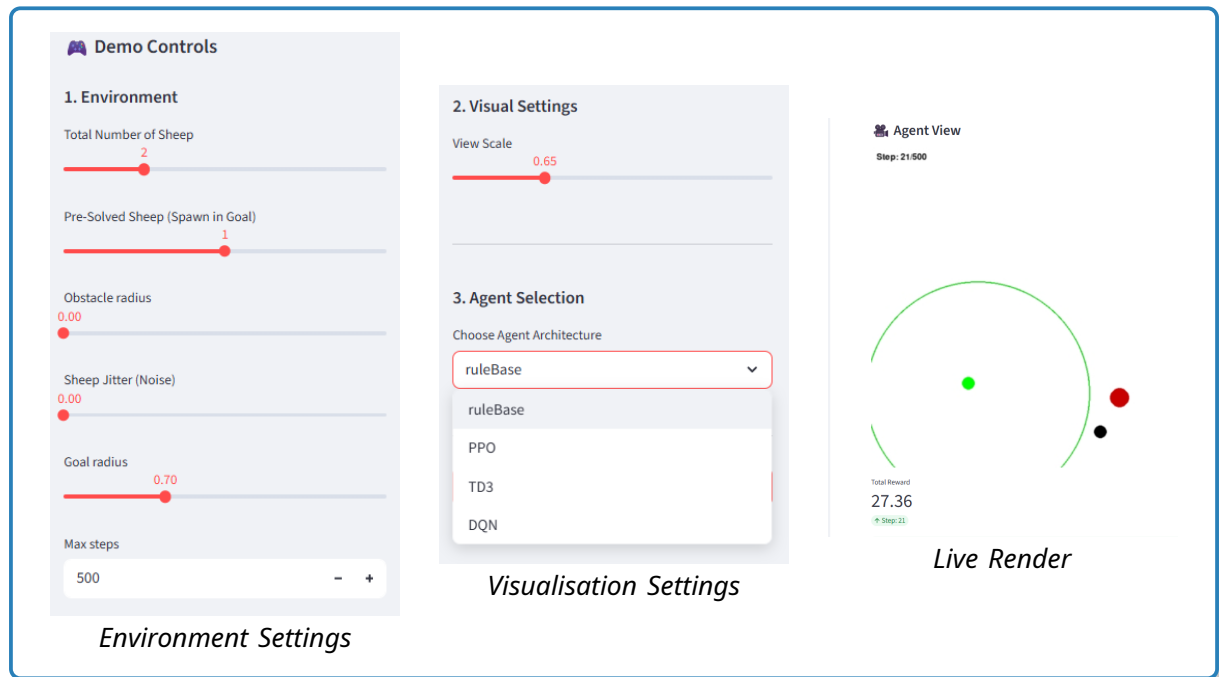


Figure 7.2: The Diagnostic Demo: (left) live Pygame environment render, (center) real-time reward breakdown, and (right) dynamic mathematical feedback.

7.2.3 Module 3: Automated Benchmarking Suite

To gather the empirical data presented in this report, manual testing is insufficient. The Benchmarking Suite allows users to queue multiple agents (both heuristic baselines and trained neural networks) and evaluate them autonomously over a specified number of episodes ($N = 500$).

It outputs a styled, color-coded tabular summary of Success Rates, Timeouts, and Average Rewards, directly facilitating the scientific comparison required for this research.

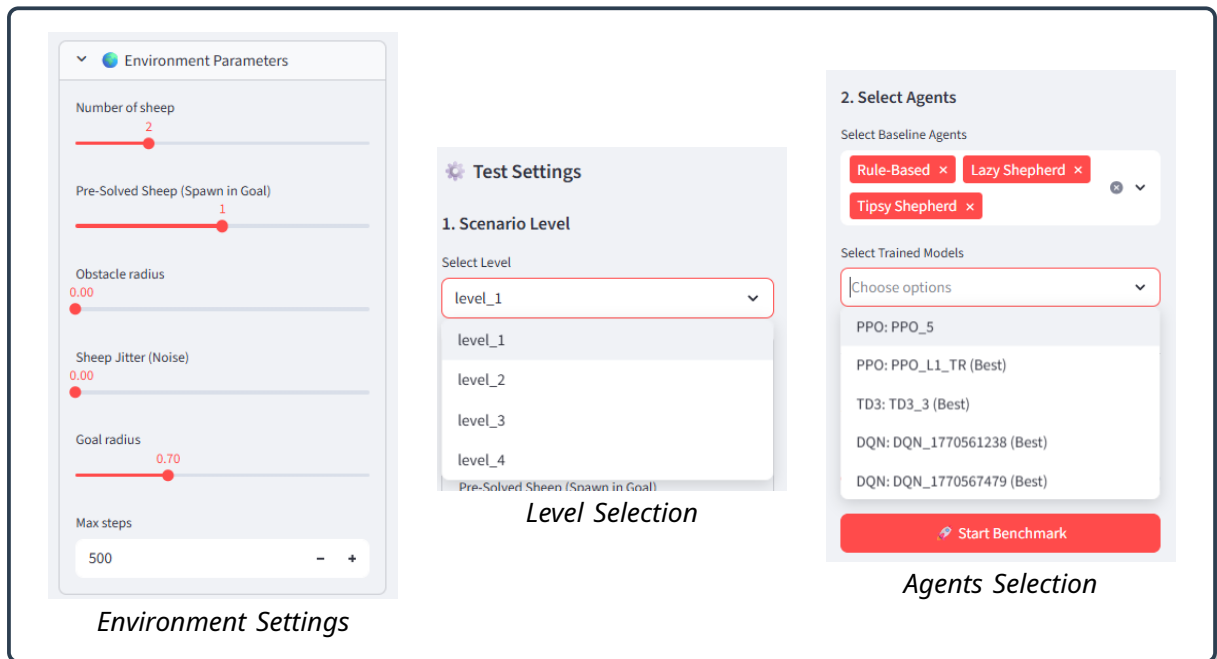


Figure 7.3: The Automated Benchmarking Suite: from multi-agent selection (left) to the final generated performance table (right).

General Conclusion

The Shepherd RL project set out to solve a challenging continuous control problem: autonomous multi-agent herding. Through systematic experimentation, we showed that applying standard reinforcement learning algorithms to sparse-reward, highly correlated environments is not sufficient on its own. Reliable autonomous behavior only emerges when strong algorithmic choices are combined with careful environment design.

Our experiments clearly identified Proximal Policy Optimization (PPO) as the most stable and effective algorithm for this task. While TD3 struggled in the sparse-reward setting and the image-based DQN faced limitations in preserving the Markov property, PPO maintained stable learning dynamics and smoothly transitioned from exploration to confident task execution. This established a solid foundation for progressively increasing task complexity.

A major contribution of this work was diagnosing and eliminating reward-hacking behaviors. The agent initially exploited weaknesses in the reward structure, leading to inefficient behaviors such as vibrating in place or endlessly circling the flock. Rather than relying solely on additional penalties, we redesigned the environment itself through Physics Locking, which constrained already secured targets. This structural modification aligned the agent's incentives with the true objective and eliminated pathological strategies, forcing genuine task completion.

We also demonstrated the critical importance of Curriculum Learning. Directly training in the most complex environment resulted in near-total failure, confirming that gradual exposure to difficulty is essential in multi-agent control. To address the challenges of scaling from one to multiple sheep, we introduced a "Pre-Solved" initialization strategy that preserved a consistent observation structure while progressively increasing complexity. This approach significantly improved performance in the most chaotic environment and validated the effectiveness of staged learning.

In conclusion, this project highlights a fundamental principle of deep reinforcement learning: agents optimize exactly what their environment allows. Success is not determined by algorithms alone, but by how well the learning dynamics, reward structure, and environmental constraints are aligned with the intended objective. Mastering the task ultimately required mastering the environment itself, laying a strong foundation for future optimization and continued performance improvements.

Bibliography

- [1] Schulman, J., Wolski, F., Dhariwal, P., Radford, A., & Klimov, O. (2017). *Proximal Policy Optimization Algorithms*. arXiv preprint arXiv:1707.06347.
- [2] Fujimoto, S., Hoof, H., & Meger, D. (2018). *Addressing Function Approximation Error in Actor-Critic Methods*. In International Conference on Machine Learning (pp. 1587-1596).
- [3] Mnih, V., Kavukcuoglu, K., Silver, D., et al. (2015). *Human-level control through deep reinforcement learning*. Nature, 518(7540), 529-533.
- [4] Bengio, Y., Louradour, J., Collobert, R., & Weston, J. (2009). *Curriculum learning*. In Proceedings of the 26th annual international conference on machine learning (pp. 41-48).
- [5] Taylor, M. E., & Stone, P. (2009). *Transfer Learning for Reinforcement Learning Domains: A Survey*. Journal of Machine Learning Research, 10(7).
- [6] Akiba, T., Sano, S., Yanase, T., Ohta, T., & Koyama, M. (2019). *Optuna: A Next-generation Hyperparameter Optimization Framework*. In Proceedings of the 25th ACM SIGKDD International Conference on Knowledge Discovery & Data Mining (pp. 2623-2631).